# BÀI TẬP CHƯƠNG 11

**Câu 1:** Hãy xây dựng ứng dụng WinForms **"Sổ chi tiêu cá nhân"** (EF Core Database First) để ghi lại từng khoản chi trong ngày, tra cứu lại theo loại chi hoặc theo khoảng thời gian, và xem tổng tiền đã tiêu. Sau khi hoàn thành, hãy đóng gói để chép sang laptop của người dùng khác và chạy được mà không phải sửa code.

**Cơ sở dữ liệu gợi ý (SQL Server – tên CSDL: QL_ChiTieu)**

| Bảng | Các cột gợi ý (khóa chính in đậm) |
|---|---|
| **ChiTieu** | **MaCT** (int, identity, PK) · NgayChi (date) · NoiDung (nvarchar 100) · LoaiChi (nvarchar 30) · SoTien (decimal 18,0) · GhiChu (nvarchar 200, cho phép null) |

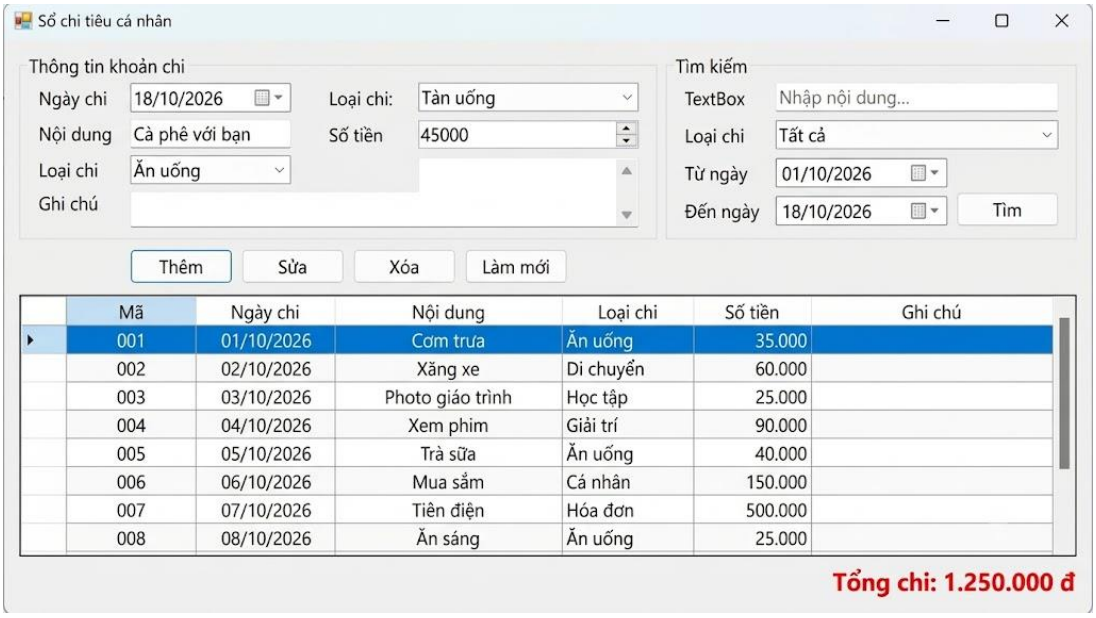
**Các task cần thực hiện**

1. **Tạo CSDL và scaffold:** tạo CSDL, bảng ChiTieu, nhập sẵn ít nhất 10 dòng dữ liệu mẫu (ăn uống, đi lại, học tập, giải trí…); dùng EF Core Database First sinh DbContext và Model.
2. **Thiết kế FormChiTieu:** DateTimePicker (ngày chi), TextBox (nội dung), ComboBox (loại chi: Ăn uống, Đi lại, Học tập, Giải trí, Khác), NumericUpDown (số tiền, bước nhảy 1.000), TextBox nhiều dòng (ghi chú), DataGridView (danh sách), các nút Thêm / Sửa / Xóa / Làm mới.
3. **CRUD một bảng:** bấm vào một dòng trong DataGridView thì dữ liệu hiện lên các control để sửa; kiểm tra hợp lệ (nội dung không rỗng, số tiền > 0, ngày chi không lớn hơn hôm nay); hỏi xác nhận trước khi xóa.
4. **Tìm kiếm / lọc:** ô tìm theo nội dung, ComboBox lọc theo loại chi, hai DateTimePicker "Từ ngày – Đến ngày"; Label **"Tổng chi: … đ"** tự cập nhật theo kết quả đang hiển thị.
5. **Cấu hình (9.2):** chuyển connection string ra file appsettings.json (thêm appsettings.Development.json để ghi đè khi lập trình), đọc bằng ConfigurationBuilder và gom vào một lớp cấu hình có kiểu (strongly-typed); đặt *Copy to Output Directory = Copy if newer*; thêm file cấu hình chứa mật khẩu vào .gitignore và tạo kèm file mẫu appsettings.example.json.
6. **Ghi log (9.3):** dùng Serilog ghi file theo ngày vào thư mục Logs cạnh file .exe; ghi mức Information khi Thêm/Sửa/Xóa (kèm MaCT), mức Error khi truy cập CSDL bị lỗi (bọc try/catch và báo cho người dùng một thông báo thân thiện).
7. **Đóng gói (9.1):** publish dạng Framework-Dependent, win-x64; chép thư mục publish sang máy khác đã cài .NET Desktop Runtime, sửa connection string trong appsettings.json cho đúng máy đó (không build lại) rồi chạy thử.

**Kết quả mong đợi**

- Form hiển thị danh sách chi tiêu; thêm/sửa/xóa xong lưới tự làm mới và dữ liệu đúng trong CSDL.

- Lọc “Ăn uống” trong tháng này chỉ còn các dòng phù hợp, Label tổng chi khớp với tổng thủ công.
- Đổi connection string trong appsettings.json thì ứng dụng kết nối sang CSDL khác mà không cần build lại.
- Thư mục Logs có file log-yyyyMMdd.txt ghi lại các thao tác; ứng dụng publish chạy được trên máy thứ hai.



**Câu 2:** Bạn hãy xây dựng ứng dụng **"Quản lý nhà trọ"** gồm bảng phòng trọ và bảng khách thuê (liên kết nhau), trong đó quê quán của khách được chọn từ **danh sách Tỉnh/Thành → Phường/Xã lấy trực tiếp từ API hành chính Việt Nam**. Ứng dụng phải "không chết" khi mất mạng hoặc gặp lỗi bất ngờ, vì cô Lan không rành công nghệ.

**Cơ sở dữ liệu gợi ý (tên CSDL: QL_NhaTro)**

| Bảng | Các cột gợi ý (khóa chính in đậm) |
|---|---|
| **PhongTro** | **MaPhong** (int, identity, PK) · TenPhong (nvarchar 20) · GiaThue (decimal 18,0) · TinhTrang (nvarchar 20: "Trống" / "Đang thuê") |
| **KhachThue** | **MaKhach** (int, identity, PK) · HoTen (nvarchar 100) · GioiTinh (nvarchar 5) · NgaySinh (date) · SoDienThoai (varchar 15) · MaTinh (int) · TenTinh (nvarchar 100) · MaPhuongXa (int) · TenPhuongXa (nvarchar 100) · DaDangKyTamTru (bit) · **MaPhong** (int, FK → PhongTro) |

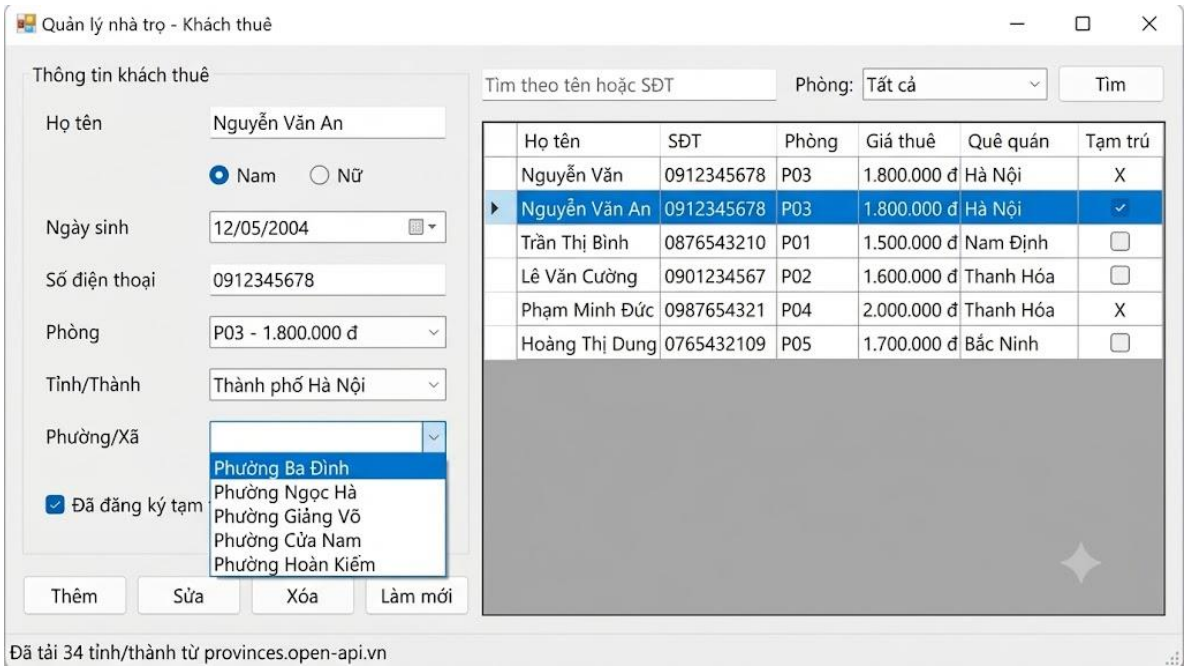
**Các task cần thực hiện**

1. **Tạo CSDL và scaffold:** tạo 2 bảng có khóa ngoại, nhập sẵn 6 phòng và vài khách; scaffold bằng EF Core Database First (kiểm tra navigation property giữa hai bảng).
2. **Thiết kế FormKhachThue:** TextBox (họ tên), hai RadioButton Nam/Nữ trong GroupBox, DateTimePicker (ngày sinh), TextBox (số điện thoại), ComboBox (phòng), ComboBox "Tỉnh/Thành", ComboBox "Phường/Xã", CheckBox "Đã đăng ký tạm trú", DataGridView, các nút Thêm / Sửa / Xóa / Làm mới.
3. **Tích hợp API Tỉnh/Phường-Xã (9.4):** khi form mở, gọi bất đồng bộ GET https://provinces.open-api.vn/api/v2/ để nạp danh sách tỉnh/thành vào ComboBox; khi chọn một tỉnh, gọi GET https://provinces.open-api.vn/api/v2/p/{code}?depth=2 để nạp danh sách phường/xã của tỉnh đó. Dùng một HttpClient dùng chung, Timeout 10 giây, async/await (giao diện không bị đơ), gán DisplayMember = "name", ValueMember = "code"; trong lúc chờ hiện chữ "Đang tải…" và khóa ComboBox.
4. **CRUD trên nhiều bảng:** thêm khách (lưu cả mã và tên tỉnh, phường/xã), sửa, xóa. Khi thêm khách vào phòng thì phòng chuyển "Đang thuê"; khi xóa người cuối cùng của phòng thì phòng trở lại "Trống". ComboBox phòng khi thêm mới chỉ liệt kê các phòng hợp lệ theo quy tắc trên.
5. **Hiển thị dữ liệu liên kết:** DataGridView hiển thị kết quả JOIN gồm Họ tên, SĐT, Tên phòng, Giá thuê, Quê quán (Phường/Xã – Tỉnh), Tạm trú (CheckBoxColumn).
6. **Tìm kiếm:** TextBox tìm theo họ tên hoặc số điện thoại; ComboBox lọc theo phòng; ComboBox lọc theo tỉnh; có nút "Bỏ lọc".
7. **Xử lý lỗi toàn cục (9.3):** đăng ký Application.SetUnhandledExceptionMode, Application.ThreadException và AppDomain.CurrentDomain.UnhandledException; mọi lỗi chưa bắt đều ghi log (Serilog) và hiện thông báo thân thiện "Đã xảy ra lỗi, thông tin đã được ghi lại". Thêm một nút ẩn "Gây lỗi thử" để kiểm tra.

8. **Chịu lỗi mạng:** bắt HttpRequestException / TaskCanceledException; khi API không phản hồi thì báo "Không tải được danh sách địa chỉ, vui lòng kiểm tra mạng", ghi log cảnh báo và cho phép gõ tay Phường/Xã để vẫn lưu được khách.

**Kết quả mong đợi**

- Mở form, ComboBox Tỉnh/Thành tự đầy; chọn "Thành phố Hà Nội" thì ComboBox Phường/Xã hiện danh sách phường/xã tương ứng.
- Lưu khách xong, lưới hiển thị đúng tên phòng và quê quán; trạng thái phòng thay đổi đúng.
- Tắt mạng rồi mở form: ứng dụng không văng, có thông báo thân thiện và vẫn nhập tay được.
- Bấm "Gây lỗi thử": hiện hộp thoại thân thiện, ứng dụng vẫn chạy tiếp, file log có đầy đủ stack trace.



**Câu 3:** Bạn hãy xây dựng ứng dụng **"Kế hoạch dã ngoại"** để ghi các chuyến đi dự định và **tự tra dự báo thời tiết của địa điểm vào đúng ngày đi** thông qua API thời tiết miễn phí. Ứng dụng cần dùng được bằng cả **tiếng Việt và tiếng Anh** vì trong nhóm có bạn người nước ngoài.

**Cơ sở dữ liệu gợi ý (tên CSDL: QL_DaNgoai)**

| Bảng | Các cột gợi ý (khóa chính in đậm) |
|---|---|
| **ChuyenDi** | **MaChuyen** (int, identity, PK) · TenChuyen (nvarchar 100) · DiaDiem (nvarchar 100) · NgayDi (date) · SoNguoi (int) · NganSach (decimal 18,0) · PhuongTien (nvarchar 30) · DaHoanThanh (bit) · GhiChu (nvarchar 200, null) |

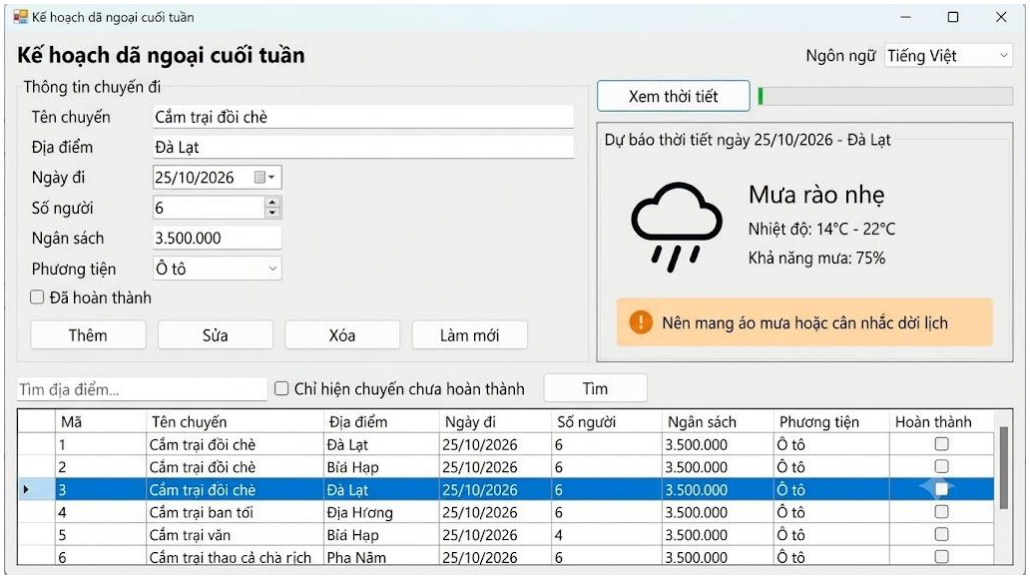
**Các task cần thực hiện**

1. **Tạo CSDL và scaffold:** tạo bảng ChuyenDi, nhập sẵn 8 chuyến (Đà Lạt, Sa Pa, Vũng Tàu, Hội An…) với ngày đi trải từ vài ngày đến vài tuần tới; scaffold bằng EF Core Database First.
2. **Thiết kế FormChuyenDi:** TextBox (tên chuyến, địa điểm), DateTimePicker (ngày đi), NumericUpDown (số người), NumericUpDown hoặc TextBox (ngân sách), ComboBox (phương tiện: Xe máy, Ô tô, Xe khách, Tàu hỏa), CheckBox "Đã hoàn thành", DataGridView, các nút Thêm / Sửa / Xóa / Làm mới.
3. **CRUD một bảng:** thêm/sửa/xóa chuyến đi; kiểm tra hợp lệ (ngày đi không nhỏ hơn hôm nay khi thêm mới, số người ≥ 1, ngân sách ≥ 0).
4. **Tìm kiếm / lọc:** TextBox tìm theo tên chuyến hoặc địa điểm, CheckBox "Chỉ hiện chuyến chưa hoàn thành", khoảng ngày đi (Từ – Đến).
5. **Tích hợp API thời tiết (9.4):** thêm nút "Xem thời tiết" (hoặc tự chạy khi chọn dòng). Bước 1: gọi https://geocoding-api.open-meteo.com/v1/search?name={DiaDiem}&count=1&language=vi để lấy latitude, longitude. Bước 2: gọi https://api.open-meteo.com/v1/forecast?latitude=…&longitude=…&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto&forecast_days=16. Trong mảng daily.time tìm đúng chỉ số của NgayDi rồi lấy các giá trị cùng chỉ số.
6. **Hiển thị kết quả:** một Panel gồm biểu tượng thời tiết (đổi theo weather_code: 0 = Trời quang; 1–3 = Có mây; 45, 48 = Sương mù; 51–67 và 80–82 = Mưa; 95–99 = Dông), nhiệt độ thấp nhất – cao nhất (°C), xác suất mưa (%); nếu xác suất mưa ≥ 60% thì tô nền cam và hiện lời khuyên "Nên mang áo mưa hoặc cân nhắc dời lịch". Ngày đi quá 16 ngày thì báo "Chưa có dữ liệu dự báo".
7. **Gọi API đúng cách:** dùng async/await, HttpClient dùng chung với Timeout 8 giây; khóa nút và hiện ProgressBar kiểu Marquee lúc đang tải; xử lý trường hợp không tìm thấy địa danh (kết quả trống – gợi ý thử tên không dấu như "Da Lat") và lỗi mạng.
8. **Đa ngôn ngữ (9.4):** thêm ComboBox hoặc hai RadioButton "Tiếng Việt / English"; khi đổi, toàn bộ nhãn, tiêu đề cột lưới, nút, thông báo và mô tả thời tiết đổi ngay (không cần

khởi động lại); định dạng ngày và tiền theo CultureInfo tương ứng; lưu lựa chọn vào usersettings.json trong %LocalAppData% để lần sau mở ứng dụng tự áp dụng.

**Kết quả mong đợi**

- Chọn chuyến "Đà Lạt – 25/10" rồi bấm "Xem thời tiết" thì thấy nhiệt độ, xác suất mưa và mô tả đúng ngày đi, khung cảnh báo xuất hiện khi khả năng mưa cao.
- Lọc theo địa điểm, ngày đi hoặc trạng thái cho ra đúng danh sách.
- Chọn English thì giao diện chuyển sang tiếng Anh ngay lập tức; đóng và mở lại ứng dụng vẫn giữ ngôn ngữ đã chọn.
- Rút dây mạng: ứng dụng báo lỗi nhẹ nhàng, không bị treo.



**Câu 4:** Hãy xây dựng ứng dụng **"Bán hàng quán trà sữa"**: quản lý thực đơn (món thuộc danh mục), lập đơn cho khách, và với mỗi đơn **tự tạo mã QR chuyển khoản VietQR** đúng số tiền và nội dung để khách quét. Thông tin tài khoản nhận tiền của quán không được viết cứng trong code.

**Cơ sở dữ liệu gợi ý (tên CSDL: QL_TraSua)**

| Bảng | Các cột gợi ý (khóa chính in đậm) |
|---|---|
| **DanhMucMon** | **MaDM** (int, identity, PK) · TenDM (nvarchar 50) – ví dụ: Trà sữa, Trà trái cây, Topping, Đồ ăn vặt |
| **MonUong** | **MaMon** (int, identity, PK) · TenMon (nvarchar 100) · GiaBan (decimal 18,0) · DangBan (bit) · **MaDM** (FK → DanhMucMon) |
| **HoaDon** | **MaHD** (int, identity, PK) · NgayLap (datetime) · TongTien (decimal 18,0) · HinhThucTT (nvarchar 20: "Tiền mặt" / "Chuyển khoản") · DaThanhToan (bit) |
| **ChiTietHoaDon** | **MaHD** (FK) + **MaMon** (FK) là khóa chính kép · SoLy (int) · DonGia (decimal 18,0) |

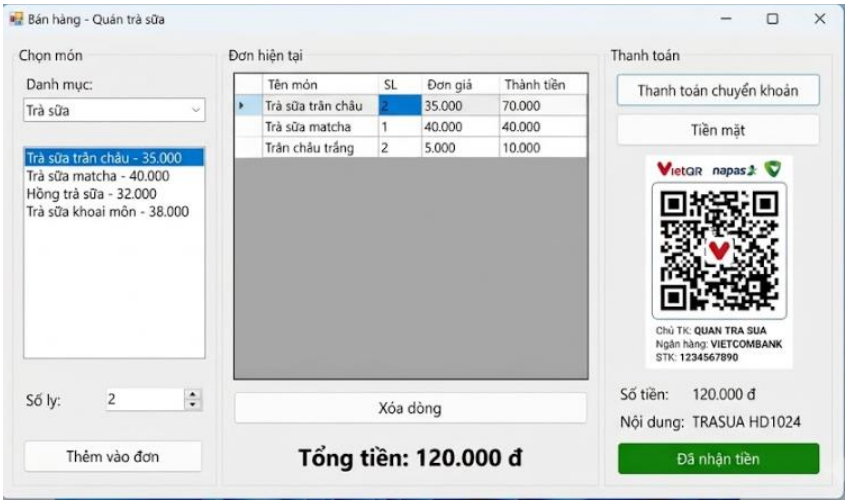
**Các task cần thực hiện**

1. **Tạo CSDL và scaffold:** tạo 4 bảng có khóa ngoại, nhập sẵn 4 danh mục và ít nhất 12 món; scaffold bằng EF Core Database First.
2. **FormThucDon (CRUD + tìm kiếm trên 2 bảng):** TextBox (tên món), ComboBox (danh mục – nạp từ bảng DanhMucMon), NumericUpDown (giá bán), CheckBox "Đang bán", DataGridView hiển thị món kèm **tên danh mục** (join); thêm / sửa / xóa món; tìm theo tên món và lọc theo danh mục.
3. **FormBanHang (lập đơn):** ComboBox chọn danh mục → ListBox (hoặc DataGridView) liệt kê các món đang bán của danh mục đó; NumericUpDown chọn số ly; nút "Thêm vào đơn" đưa món vào DataGridView giỏ hàng gồm Tên món, SL, Đơn giá, Thành tiền (cộng dồn số lượng nếu món đã có); nút "Xóa dòng"; Label tổng tiền cập nhật tức thì.
4. **Cấu hình thanh toán (9.2):** tạo mục "Payment" trong appsettings.json gồm BankId, AccountNo, AccountName, Template (ví dụ compact2) và tên quán; đọc vào lớp cấu hình có kiểu, không hard-code trong code. Khi làm bài và nộp bài hãy dùng tài khoản giả hoặc tài khoản của chính mình, không đưa số tài khoản thật của người khác lên GitHub.
5. **Thanh toán bằng mã QR (9.4):** nút "Thanh toán chuyển khoản" ghép địa chỉ https://img.vietqr.io/image/{BankId}-{AccountNo}-{Template}.png?amount={TongTien}&addInfo={NoiDung}&accountName={AccountName} (mã hóa các tham số bằng Uri.EscapeDataString; nội dung chuyển khoản ví dụ "TRASUA HD{MaHD}"); tải ảnh bằng HttpClient.GetByteArrayAsync (async/await), dựng MemoryStream rồi gán vào PictureBox.Image; hiển thị kèm số tiền và nội dung chuyển khoản dạng chữ. Nút "Đã nhận tiền" lưu HoaDon + ChiTietHoaDon trong cùng một lần SaveChanges; nút "Tiền mặt" lưu hóa đơn không cần QR.

6. **Danh sách hóa đơn:** DataGridView hóa đơn (Mã HD, giờ lập, hình thức, tổng tiền); chọn một hóa đơn thì lưới thứ hai hiện chi tiết các món; DateTimePicker lọc theo ngày và Label "Doanh thu ngày: … đ".
7. **Ghi log và chịu lỗi (9.3):** Serilog ghi Information khi tạo QR (mã HD, số tiền – không ghi số tài khoản đầy đủ), Error khi tải ảnh QR thất bại; nếu mất mạng thì vẫn hiện số tài khoản và nội dung chuyển khoản dạng chữ để khách chuyển tay, đồng thời cho phép chuyển sang thanh toán tiền mặt.

**Kết quả mong đợi**

- Chọn món, thêm vào giỏ: tổng tiền chạy đúng; bấm "Thanh toán chuyển khoản" thì trong PictureBox hiện mã VietQR có sẵn số tiền và nội dung; quét bằng app ngân hàng thấy đúng số tiền, đúng tên tài khoản.
- Bấm "Đã nhận tiền": hóa đơn xuất hiện trong danh sách, chi tiết khớp giỏ hàng, doanh thu ngày tăng đúng.
- Đổi thông tin ngân hàng trong appsettings.json thì mã QR tạo mới dùng tài khoản mới mà không phải sửa code.
- Lưu ý: VietQR chỉ tạo mã chuyển khoản, **không tự xác nhận** đã nhận tiền – thu ngân phải kiểm tra tin nhắn ngân hàng rồi mới bấm "Đã nhận tiền".



**Câu 5:** Bạn hãy xây dựng ứng dụng **"Quản lý đặt tour"** gồm 3 bảng (tour – khách hàng – đặt tour), **tự lấy tỷ giá hiện hành từ API** để quy đổi giá ra VND, và cuối cùng đóng gói thành bộ cài chạy được trên máy tính nhân viên **chưa cài .NET**.

**Cơ sở dữ liệu gợi ý (tên CSDL: QL_DatTour)**

| Bảng | Các cột gợi ý (khóa chính in đậm) |
|---|---|
| **Tour** | **MaTour** (int, identity, PK) · TenTour (nvarchar 100) · DiemDen (nvarchar 50) · NgoaiTe (char 3: THB, KRW, JPY, USD) · GiaNgoaiTe (decimal 18,2) · NgayKhoiHanh (date) · SoChoToiDa (int) |
| **KhachHang** | **MaKH** (int, identity, PK) · HoTen (nvarchar 100) · SoDienThoai (varchar 15) · Email (nvarchar 100) |
| **DatTour** | **MaDat** (int, identity, PK) · **MaKH** (FK) · **MaTour** (FK) · SoNguoi (int) · NgayDat (datetime) · TyGia (decimal 18,4) · TongTienVND (decimal 18,0) |

**Các task cần thực hiện**

1. **Tạo CSDL và scaffold:** tạo 3 bảng có khóa ngoại, nhập sẵn 5 tour (ví dụ: Bangkok – Pattaya giá THB, Seoul – Nami giá KRW, Tokyo – Phú Sĩ giá JPY) và vài khách hàng; scaffold bằng EF Core Database First.
2. **Thiết kế giao diện bằng TabControl** gồm 3 tab: "Tour" (CRUD + tìm theo tên/điểm đến; ComboBox ngoại tệ, DateTimePicker khởi hành, NumericUpDown giá và số chỗ), "Khách hàng" (CRUD + tìm theo tên/SĐT), "Đặt tour".
3. **Tab "Đặt tour":** ComboBox khách hàng, ComboBox tour, NumericUpDown số người (không vượt quá số chỗ còn lại = SoChoToiDa trừ tổng số người đã đặt tour đó), Label hiển thị giá/người bằng ngoại tệ, tỷ giá, giá/người và tổng tiền bằng VND; nút "Đặt tour" lưu vào bảng DatTour kèm **tỷ giá tại thời điểm đặt** (để sau này tỷ giá đổi thì đơn cũ không bị đổi theo).
4. **Tích hợp API tỷ giá có dự phòng (9.4):** viết TyGiaService.LayTyGiaAsync(string ngoaiTe) gọi https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currencies/{code}.json (code viết thường, ví dụ thb); kết quả dạng {"date":"…","thb":{"vnd":…}} nên lấy giá trị ở thb → vnd. Nếu địa chỉ này lỗi, tự động thử địa chỉ dự phòng https://latest.currency-api.pages.dev/v1/currencies/{code}.json; cả hai đều lỗi mới báo người dùng. Thêm bộ nhớ đệm (cache) trong bộ nhớ 10 phút để đổi qua lại giữa các tour không gọi API lặp lại.
5. **Cấu hình và log (9.2, 9.3):** đưa hai địa chỉ API và thời gian chờ vào mục "ExchangeRate" trong appsettings.json, đọc bằng lớp cấu hình có kiểu; Serilog ghi Warning khi địa chỉ chính lỗi (kèm lý do), ghi Information cho biết lần gọi này dùng địa chỉ nào; thêm bắt lỗi toàn cục (ThreadException, UnhandledException).
6. **Danh sách đặt tour (nhiều bảng):** DataGridView hiển thị JOIN gồm Họ tên khách, Tên tour, Số người, Tỷ giá, Tổng tiền VND, Ngày đặt; tìm theo tên khách, tên tour hoặc tháng khởi hành; Label tổng doanh thu của các dòng đang hiển thị.

7. **Đóng gói và triển khai (9.1):** đặt phiên bản 1.0.0 và hiển thị trên thanh tiêu đề; publish **Self-Contained, win-x64, PublishSingleFile** (kèm IncludeNativeLibrariesForSelfExtract); bảo đảm appsettings.json được chép vào thư mục publish; chép sang một máy chưa cài .NET (máy khác, máy ảo hoặc Windows Sandbox) rồi chạy thử. Lập bảng so sánh dung lượng và số file giữa bản Framework-Dependent và Self-Contained, kèm 3 ảnh chụp: thư mục publish, thông tin dung lượng, ứng dụng chạy trên máy thứ hai. Nộp kèm file script .sql tạo CSDL và dữ liệu mẫu.

**Kết quả mong đợi**

- Chọn tour Bangkok (THB) thì Label hiện tỷ giá THB → VND hôm nay và tổng tiền VND = giá × số người × tỷ giá; chọn tour Seoul (KRW) thì tỷ giá đổi theo.
- Cố tình sai địa chỉ chính trong appsettings.json: ứng dụng vẫn lấy được tỷ giá nhờ địa chỉ dự phòng, file log ghi rõ đã chuyển sang địa chỉ dự phòng.
- Đặt vượt số chỗ còn lại thì bị chặn kèm thông báo; đơn đã lưu giữ nguyên tỷ giá cũ.
- Bản publish Self-Contained chạy được trên máy chưa cài .NET sau khi chỉnh connection string trong appsettings.json.

